



LCI Advanced Workshop 2025: Introductions

Willy Dizon
Senior Systems Analyst
Arizona State University
willy.dizon@asu.edu

*This document is a result of work by volunteer LCI instructors and is licensed under CC BY-NC 4.0
(<https://creativecommons.org/licenses/by-nc/4.0/>)*



- Your name
- Institution you are from
- How long you have been using Slurm
- What version are you using
- How large is your cluster
- What is your biggest pain point with Slurm?



Join at menti.com | use code **4212 9178**



LCI Advanced Workshop 2025: **Building Slurm / Upgrading**

Willy Dizon
Senior Systems Analyst
Arizona State University
willy.dizon@asu.edu

*This document is a result of work by volunteer LCI instructors and is licensed under CC BY-NC 4.0
(<https://creativecommons.org/licenses/by-nc/4.0/>)*

Git Repository

These slides can be found online at:

<https://github.com/yesthatwilly/slurm-quickstart>

Building and Upgrading Overview

- Introduction and Objectives
 - Update Frequency and Versioning
 - What is the Goal?
- Best Practices
 - Backup your MySQL DB
 - Backup your jobqueue / controller state
 - Compiling Slurm
 - Update the database
 - Restore all Services

Update Frequency and Versioning

Upgrades to Slurm become available [every six months](#) and follow the naming convention:

YY.MM.R

- 2 digit year
- 2 digit month
- 1 digit maintenance version

Upgrades should happen cluster-wide, for the head node and **all** compute nodes.

Do not mix and match versions, and upgrade in smaller increments if you fall behind more than 2 releases.

What is the Goal?

Have a highly-reproducible build system that ensures immediate consistency among all Slurm nodes.

Possible Upgrade Methods:

- built into a deployed image
- installed via RPM
- compiled and shared onto a network filesystem
- manually and remotely deployed to nodes

Best Practices

Slurm upgrades are [highly documented](#) already.

1. Backup your database
2. Backup your jobqueue
3. Compile to a versioned directory
4. Complete the upgrade
5. Keep old builds available

Best Practices (cont.)

1 - Backup your MySQL database

```
mysqldump -u slurm -p --databases slurm_acct_db > backup.sql
```

Do this before an upgrade—also consider doing this routinely with crontabs:

```
0 2 * * * mysqldump -u slurm -p"lciadvPW" --single-transaction --databases  
slurm_acct_db > /path/to/nfs/backup-$(date +%F).sql
```

This can produce dated files in the form of **backup-2025-10-10.sql** and produces one backup file every 2:00AM. Choose your backup schedule as per your needs and at a time of relatively low database activity.

1 - Backup your MySQL database

Passwords for the database user can alternatively (and is recommended!) to be saved in `~/.my.cnf`.

```
[rocky@lci-head-02-1 ~]$ cat ~/.my.cnf
[client]
user=slurm
password=lciadvPW
```

That will permit you to simplify the command to:

```
0 2 * * * mysqldump --single-transaction --databases
slurm_acct_db > /path/to/nfs/backup-$(date +%F).sql
```

1 - Backup your MySQL database

Test the command:

```
$ mysqldump --single-transaction --databases slurm_acct_db >  
/home/rocky/sql_backups/backup-$(date +%Y-%m-%d_%H-%M).sql  
  
$ ls ~/sql_backups/  
backup-2025-10-10_14-56.sql
```

1 - Backup your MySQL database

Take note, **any** user can be used to create these backups, since this is a database credential and not a POSIX user.

`rocky` is convenient for the lab, but may be wholly unsuitable in your own site.

Consider:

- Running backups a remote host to both `slurmctld` and `mysqldb`
- Creating a separate `mysqld` credential with read-only access to **`slurm_acct_db`**
- Keeping the backups off local storage, e.g., a network fileshare
- Using a database backup suite, entirely sidestepping manual backups

1 - Backup your MySQL database

The prescribed order:

```
# systemctl stop slurmctld  
# sacctmgr shutdown  
# systemctl stop slurmdbd  
# systemctl status slurmdbd  
# mysqldump ...
```

mysqldump after the slurmdbd service is down; this ensures that you have captured all data generated since the service shutdown.

1 - Backup your MySQL database

Ensure proper service closure with `sacctmgr shutdown`

```
[root@lci-head-02-1 ~]# sacct
```

JobID	JobName	Partition	Account	AllocCPUS	State	ExitCode
1	interacti+	general	root	1	COMPLETED	0:0
1.interacti+	interacti+		root	1	COMPLETED	0:0
1.extern	extern		root	1	COMPLETED	0:0

```
[root@lci-head-02-1 ~]# sacctmgr shutdown
sacctmgr: error: _open_persist_conn: failed to open persistent connection to host:lci-head-02-1:6819:
Connection refused
sacctmgr: error: Getting response to message type: DBD_SHUTDOWN
[root@lci-head-02-1 ~]# sacct
sacct: error: _open_persist_conn: failed to open persistent connection to
host:lci-head-02-1:6819: Connection refused
sacct: error: Sending PersistInit msg: Connection refused
sacct: error: Problem talking to the database: Connection refused
```

(IF you need to) Restore your MySQL database

Restoring the database is not necessary during upgrades! This is here for completeness, if the need arises.

Ensure the `slurmdbd` process is down before doing this!

Considering the weight of redoing the `slurm_acct_db`, it will be beneficial to be in an outage with `slurmctld` also down.

```
# systemctl stop slurmdbd
# mysql -e "DROP DATABASE slurm_acct_db;"
# mysql < /path/to/backup.sql
```

The `backup.sql` file will create the database if it does not already exist:

```
[rocky@lci-head-02-1 sql_backups]$ grep "CREATE" backup-2025-10-10_15-37.sql | head -n 1
CREATE DATABASE /*!32312 IF NOT EXISTS*/ `slurm_acct_db` /*!40100 DEFAULT CHARACTER SET
latin1 COLLATE latin1_swedish_ci */;
```

Best Practices (cont.)

2 - Backup your jobqueue

The Slurm controller state—jobs in the queue, etc.--are saved to the filesystem:

```
$ scontrol show config | grep Save
StateSaveLocation          = /var/spool/slurm.state

$ ls /var/spool/slurm.state/
assoc_mgr_state  assoc_usage.old  fed_mgr_state          job_state.old
last_tres        node_state.old   priority_last_decay_ran qos_usage.old
trigger_state    trigger_state.old  assoc_mgr_state.old
clustername      fed_mgr_state.old  last_config_lite
last_tres.old    part_state        priority_last_decay_ran.old  resv_state
assoc_usage      dbd.messages      job_state               last_config_lite.old
node_state       part_state.old     qos_usage                resv_state.old
```

2 - Backup your jobqueue

Using tar

```
$ tar -czf jobq-$(date +%F).tar.gz /var/spool/slurm.state  
$ ls jobq*  
jobq-2025-10-13.tar.gz
```

Using rsync

```
$ rsync -av /var/spool/slurm.state /tmp/slurm.state
```

If anything goes wrong during the upgrade—whether there is accidental/user-error file loss, or there is a need to return to the previous Slurm version, just repopulate the directory with your backed up files.

Best Practices (cont.)

3 - Compiling Slurm

USE A BUILDSRIPT

Ensure consistent builds with new versions. Don't rely on bash history. Don't just re-follow the manual instructions.

```
# tail -n 20 mkslurm-x86_64.sh
make clean || true
./configure \
  --without-shared-libslurm \
  --with-pmix=$PMIXPATH \
  --with-munge \
  --with-hwloc \
  --with-jwt \
  --with-json \
  --enable-slurmrestd \
  --prefix=$SLURM_DESTDIR \
  --with-ucx \
  --with-lua \
  --enable-pam \
  --with-pam_dir=$SLURM_DESTDIR/lib64/security \
  --sysconfdir=/etc/slurm
make -j $MAKE_J
```

Using a reproducible script:

- ensures explicit inclusion of required features, e.g., pmix, ucx, lua
- allows fixing of PMIX/UCX versions
- allows parallel installs for easy rollback

```
make install
make install-contrib
```

4 - Complete the Upgrade

Restart [in Order](#), interactively

```
# sudo -u slurm /path/to/slurmdbd -D
[2025-10-13T08:26:32.586] dropping column lft from table "testc_assoc_table"
[2025-10-13T08:26:32.586] dropping column rgt from table "testc_assoc_table"
...
[2025-10-13T08:27:56.564] adding key archive_delete (deleted) to table
"testc_job_table"
[2025-10-13T08:47:16.719] Warning: Note very large processing time from make table
current "testc_job_table": usec=1160154890 began=08:27:56.564
...
[2025-10-13T08:53:09.063] slurmdbd version 25.05.3 started
```

Don't restart from the unit-file.

Use `sudo` to assure long-running processes such as database schema upgrades are not interrupted by an impatient systemd timeout.

It can take a while, depending on changes to the schema for a particular slurm upgrade and scales along the size of your database (mostly jobcounts).

When the process reports “slurmdbd started”, ctrl-C the process.

4 - Complete the Upgrade

Have ample storage on SQL host

```
[2025-10-13T09:22:26.024] adding key archive_delete (deleted) to table "testc_job_table"
[2025-10-13T09:43:48.272] error: mysql_query failed: 1114The table 'testc_job_table' is full
alter table "testc_job_table" modify `job_db_inx` bigint unsigned not null auto_increment, modify
`mod_time` bigint unsigned default 0 not null, modify `deleted` tinyint default 0 not null, modify
`account` tinytext, modify `admin_comment` text, modify `array_task_str` text, modify ...
[2025-10-13T09:43:48.272] accounting_storage/as_mysql: init: Accounting storage MYSQL plugin failed
[2025-10-13T09:43:48.356] error: Couldn't load specified plugin name for accounting_storage/mysql:
Plugin init() callback failed
```

If you have recorded millions *or tens of millions of jobs*, disk space might be an issue during the upgrade. While it can be resumed gracefully, it feels cleaner to anticipate this need ahead of time.

The above failure required > 40GB of free space to complete the upgrade. The jobcounts were extensive due to job arrays.

4 - Complete the Upgrade

Finally, restart the services through unit-files.

On the controller:

```
# systemctl start slurmdbd
# systemctl start slurmctld
# systemctl start slurmrestd
```

On the compute nodes:

```
# systemctl start slurmd
```

Confirm the nodes are available and UP!

```
# sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
public      up 7-00:00:00    168  maint sc[005-112],sg[001-023,025-050,230-239,250]
```

5 - Keep Old Versions

```
# ls -l
24.05.0-20240624
24.05.0-20240625
24.05.0-20240626
24.05.1-20240708
24.05.1-20240906
24.05.3-20240927
24.11.3-20250320
25.05.3-20250929
25.05.3-20251001
25.05.3-20251010
current
dev

# ln -sf 25.05.3-20251010 dev
```

Not every build will be used, that's OK.

However, keeping each version:

- has negligible storage cost
- allows quick A/B testing
- easily allows rollbacks

When you've decided which build is best suited for the upgrade, it's a simple symlink update.

Also, *upgrading in increments becomes very easy* as you can build each of them in succession much like an assembly line.

Building Slurm / Upgrading

Thank you!

Any questions?

Upgrading Slurm: The Lab

This will walk you through the steps outlined in the Lecture, bringing 24.11.6 to 25.05.3.

ssh [rocky@lci-head-XX-1.ncsa.cloud](#)

Password: LCIr0cky9test!

1. [Pre-work] Getting Slurm up on our private clusters
2. Shut down slurmdbd gracefully
3. Shut down other slurm controller services
4. Backup slurm database
5. Backup jobqueue
6. Build 25.05.3
7. Update symlink for systemd
8. Restart all slurm controller services
9. Restart all slurmd compute node services
10. Test Availability

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

We have been assigned nodes with the following host machines:

lci-head-XX-1
lci-compute-XX-1
lci-compute-XX-2

where XX is our seat number. These nodes are running Rocky 9 and have all the required packages to build and operate a base version of Slurm 24.11.6.

However, due to this being installed as an identical template for all users, all machines' `slurm.conf` share the same seat and IP address, so the cluster does not work.

This exercise will give us a firm understanding of the infrastructure of our cluster and how we can navigate from node to node.

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

Let's identify our systems according to `/etc/hosts` and use this to populate our configuration.

```
$ cat /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6

141.142.217.8    lci-head-02-1.ncsa.cloud
192.168.2.151    lci-head-02-1
192.168.2.212    lci-compute-02-1
192.168.2.197    lci-compute-02-2
```

Take note of the “head” node’s 192.x IP address.

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

Using `sudo -i` will let us passwordlessly elevate to `root`.

In `/etc/slurm/slurm.conf`, identify and update the following lines:

```
SlurmctldHost=lci-head-XX-1(192.168.2.YYY)
```

```
AccountingStorageHost=lci-head-XX-1
```

```
NodeName=lci-compute-XX-[1-2] \
```

```
...
```

```
Nodes=lci-compute-XX-[1-2] \
```

Use your seat number, accordingly. Respectively, this is where to find the `slurmctld` process, the `slurmdbd` process, and then `slurmd` processes.

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

`slurm.conf` updates typically only warrant `scontrol reconfig`.

Since the cluster is down, we need a service restart:

```
# systemctl start slurmdbd
# systemctl start slurmctld
# sinfo
```

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
general	up	1-00:00:00	2	unk	lci-compute-XX-[1-2]

`sinfo` should report “unk (nown)” nodes because those each of these `slurmd` processes on the compute node are still actively seeking a controller on the incorrect host.

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

Our private cluster is set up so that the `root` user—and only from the head node—may ssh to all other compute nodes without a password.

```
[root@lci-head-02-1 ~]# ssh lci-compute-02-1  
[root@lci-compute-02-1 ~]#
```

What happens when we try to start the `slurmd` process?

```
[root@lci-compute-02-1 ~]# systemctl start slurmd
```

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

```
[root@lci-compute-02-1 ~]# systemctl start slurmd
```

It won't load. It might even stall for a full minute or longer without error.

Let's review why. `systemctl status slurmd` will tell us the reason:

```
Oct 17 18:31:34 lci-compute-02-1.novalocal slurmd[40439]: [2025-10-17T18:31:34.822]  
error: _fetch_child: failed to fetch remote configs: Unable to contact slurm  
controller (connect failure)
```

Where is `slurmd` looking for the `slurmctld` controller?

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

The systemd unit files tell us how `slurmd` is being invoked:

```
[root@lci-compute-02-1 ~]# systemctl status slurmd
```

- `slurmd.service` - Slurm node daemon

```
Loaded: loaded (/etc/systemd/system/slurmd.service; enabled; preset: disabled)
```

So let's look at the `slurmd.service` file!

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

Review the `slurmd.service` file and look for `ExecStart`:

```
[root@lci-compute-02-1 ~]# cat /etc/systemd/system/slurmd.service
[Unit]
Description=Slurm node daemon
After=munge.service network-online.target remote-fs.target
Wants=network-online.target
#ConditionPathExists=/etc/slurm/slurm.conf

[Service]
EnvironmentFile=-/etc/default/slurmd
Type=notify
RuntimeDirectory=slurm
RuntimeDirectoryMode=0755
ExecStart=/opt/slurm/current/sbin/slurmd --systemd--conf-server $SLURMCTLD_HOST $SLURMD_OPTIONS
```

It's a variable stored outside the service file, in an `EnvironmentFile`:

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

And `/etc/default/slurmd` gives us:

```
[root@lci-compute-02-1 ~]# cat /etc/default/slurmd
# /etc/default/slurmd - created by 08_deploy_compute
# Contains environment variables consumed by systemd unit EnvironmentFile
SLURMCTLD_HOST=192.168.2.152
```

Perfect, now change this to match our **lci-head-XX-1**'s IP address.

Then bring up `slurmd`:

```
[root@lci-compute-02-1 ~]# systemctl start slurmd
[root@lci-compute-02-1 ~]# systemctl status slurmd
• slurmd.service - Slurm node daemon
   Loaded: loaded (/etc/systemd/system/slurmd.service; enabled; preset: disabled)
   Active: active (running) since Fri 2025-10-17 18:40:45 CDT; 1s ago
```

Repeat these steps for both `lci-compute-XX-[1,2]`

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

Verify it is working with `sinfo`:

```
[root@lci-compute-02-1 ~]# sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
general      up 1-00:00:00      2   idle lci-compute-02-[1-2]
[root@lci-compute-02-1 ~]# salloc -p general
salloc: Granted job allocation 9
salloc: Nodes lci-compute-02-1 are ready for job
[root@lci-compute-02-1 ~]#
```

Congrats, your cluster is up and working!

Learn and use this troubleshooting path with nodes reporting ``unk(known)``, which indicates simply “slurmctld isn’t communicating with slurmd”.

Upgrading Slurm Lab

1. Getting Slurm up on our Private Clusters

This setup—**like all setups should**—relies on [configless slurm](#).

The alternative is that `/etc/slurm/slurm.conf` needs to be distributed on each change to each node. This is possible through orchestration efforts like salt or ansible, but adds more complexity compared to “ensure the compute node knows the controller’s IP”.

Now that Slurm is up, let’s get back to the head node and start the upgrade-

Upgrading Slurm Lab

2. Shut down slurmdbd gracefully

```
[root@lci-head-02-1 ~]# sacctmgr shutdown
sacctmgr: error: _open_persist_conn: failed to open persistent connection to
host:lci-head-02-1:6819: Connection refused
sacctmgr: error: Getting response to message type: DBD_SHUTDOWN
```

```
[root@lci-head-02-1 ~]# systemctl stop slurmdbd
```

Here, `sacctmgr shutdown` should suffice in taking down the process, presuming *the process is already running*. The `systemctl stop` is redundant, but the assurance we need the process is definitely down.

Upgrading Slurm Lab

3. Shut down other slurm controller services

```
# systemctl stop slurmctld  
# systemctl stop slurmrestd
```

`Slurmrestd` is not running on our current cluster, but it may be, at home. If it's running, be sure to take it down, too.

Upgrading Slurm Lab

4. Backup slurm database

```
# mysqldump --single-transaction --databases slurm_acct_db > /root/last-minute-slurm.sql
```

Credentials are required for a mysqldump, normally `-u root -p password`

We don't need it here because we're running a mysqldb in a toy lab.

Upgrading Slurm Lab

5. Backup jobqueue

```
# tar -czf /root/last-minute-jobq.tgz /var/spool/slurm.state
```

With a database backup while the slurmdbd service is down,
and
with a controller state while the slurmctld service is down,

we now have a complete, coherent snapshot that will allow us to rollback to our previous Slurm version, or to make additional attempts at upgrading if we run into any complications.

Upgrading Slurm Lab

6. Build Slurm 25.05.3

Ensure all new dependencies are installed for any additional features you wish to compile with. For a later lab, we will require Signal-to-Noise:

```
# dnf install -y s2n-tls s2n-tls-devel
# ssh lci-compute-XX-1 "dnf -yq install s2n-tls"
# ssh lci-compute-XX-2 "dnf -yq install s2n-tls"
```

This is for the security lecture and lab, regarding TLS functionality.

This is what we would see in the `./configure` step without it:

```
./configure: WARNING: unable to locate 1.5.7+ s2n library
checking for s2n installation...no
```

Other convenient helpers (but optional!)

```
# dnf install -y bash-completion
```

Upgrading Slurm Lab

6. Build Slurm 25.05.3

To simplify this lab, and to ready us for the later labs, a script is provided to complete this build. Again, for your own benefit, get accustomed to using a script for highly-reproducible builds.

```
# su - rocky
$ cd slurm-quickstart
$ bash ./04_build_slurm 25.05.3
$ sudo bash ./08_deploy_compute
```

Upgrading Slurm Lab

7. Update symlink for systemd

We should now have two slurm builds in /opt/slurm. We can compile many builds, every day for a month and it shouldn't impact our prod server, thanks to the symlink mechanism.

Now that we've landed on a build we are happy with, we can update the symlink.

```
# cd /opt/slurm
# ln -sf 25.05.3-20251017 current
# sudo -i
# ssh lci-compute-XX-1 "ln -sf /opt/slurm/25.05.3-20251020/ /opt/slurm/current"
# ssh lci-compute-XX-2 "ln -sf /opt/slurm/25.05.3-20251020/ /opt/slurm/current"
# ls -l
drwxr-xr-x. 8 rocky rocky 81 Oct 17 15:15 24.11.6-20251017
drwxr-xr-x. 8 rocky rocky 81 Oct 17 15:54 25.05.3-20251017
lrwxrwxrwx. 1 root root 16 Oct 17 16:49 current -> 25.05.3-20251017
```

Upgrading Slurm Lab

8. Restart all slurm controller services

```
# systemctl start slurmdbd  
# systemctl start slurmctld  
# systemctl start slurmrestd
```

Each of these service startups should give no output and should return immediately. Any delays or output typically suggest a problem.

Investigate any problems with `systemctl status slurmXXXX`

Verify the controller and database are online `scontrol ping ; sacctmgr ping`

Upgrading Slurm Lab

9. Restart all slurmd services

```
# systemctl stop slurmd  
# systemctl start slurmd
```

You will need to ssh to each of your compute nodes to accomplish this.

This service startup should also give no output and should return immediately.

Investigate any problems with `systemctl status slurmd`

Upgrading Slurm Lab

10. Test Availability

Switch back to `root` to test:

```
[root@lci-head-02-1 ~]# salloc -p general
salloc: Granted job allocation 1
salloc: Nodes lci-compute-02-1 are ready for job
[root@lci-compute-02-1 ~]#
```

Yatta! Allocating a job means the `slurmd` process is cooperating with the `slurmctld` process and the upgrade is successful!

```
[root@lci-compute-02-1 ~]# sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
general    up 1-00:00:00      1    mix lci-compute-02-1
general    up 1-00:00:00      1   idle lci-compute-02-2
```

Upgrading Slurm Lab

To create a 'rocky' slurm user for non-root job submissions:

With `sudo`, or as `root`:

```
# 1. Create an account (e.g., "hpc" as the top-level project/account)
/opt/slurm/current/bin/sacctmgr add account lci Description="lci hpc account" Organization="lci"

# 2. Create the user "rocky" and associate it with that account
/opt/slurm/current/bin/sacctmgr add user rocky Account=lci DefaultAccount=lci QOS=normal

# 3. (Optional) Verify the user and account association
/opt/slurm/current/bin/sacctmgr show user withassoc format=User,DefaultAccount,Account,Cluster
```